

s16 Team Protocol 学习笔记

主题：Lead / Teammate / MessageBus / request_id / pending_requests / inbox 分流

本笔记整理了本次对话中围绕 s16 协议层、团队消息系统、计划审批、shutdown、continue 语义等问题的完整脉络。

0. 一句话总览

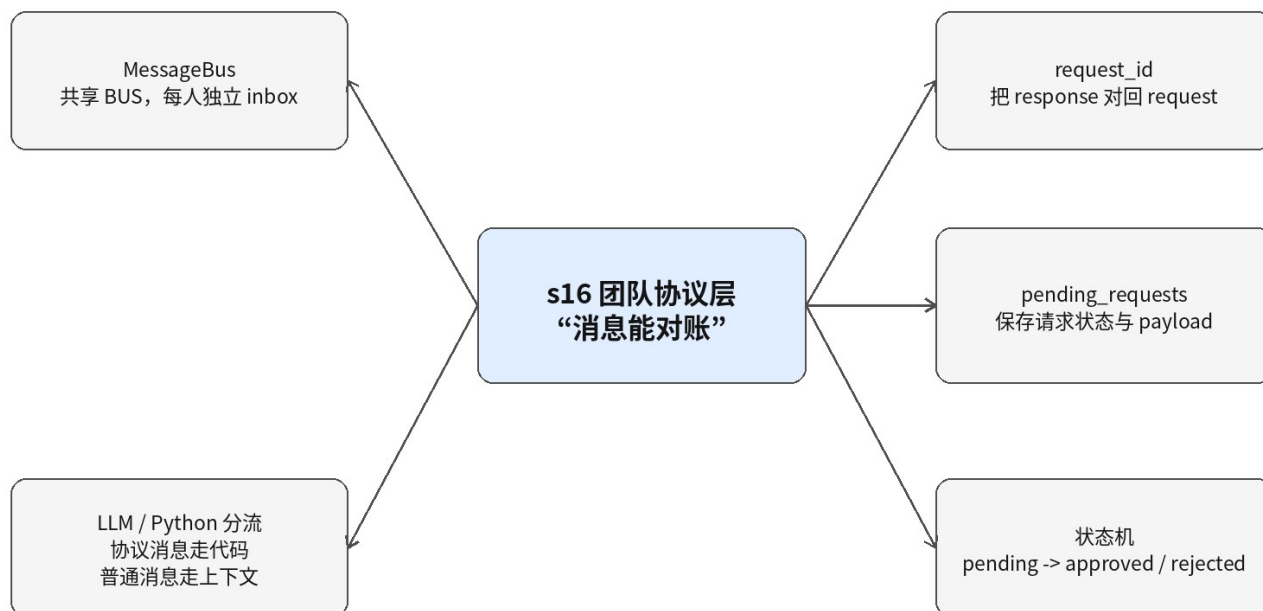
核心结论： s15 解决“消息能送到”；s16 在此基础上解决“回复能对上”。MessageBus 负责传消息，request_id 负责对账，pending_requests 负责记账，状态机负责定结果。

- **s15 的重点：** Lead 和 teammate 可以通过 MessageBus 互相发消息，各自从自己的 inbox 收消息。
- **s16 的重点：** 把普通消息升级成带 request_id 的协议消息，使 request 和 response 可以可靠匹配。
- **本次最大的易混点：** Lead 侧和 teammate 侧都在读 inbox，但它们处理的是不同方向、不同语义的消息。

1. 总体思维导图

下面的图把本次对话的几个核心概念放在一张图里：

思维导图 1：s16 团队协议层总览



2. BUS 和 inbox：共用总线，不共用同一个 inbox

重点： 整个团队共用一个 BUS / MessageBus 对象，但不是共用一个 inbox 列表。每个 agent 都有自己的 inbox。

```
BUS.inboxes = {
    "lead": [],
    "planner": [],
    "coder": [],
    "reviewer": [],
}
```

- `BUS.send("lead", "coder", ...)` 的本质是把消息追加到 `BUS.inboxes["coder"]`。
- `BUS.read\_inbox("coder")` 只读取 coder 自己的 inbox，不会读取整个团队的公共列表。
- 因此不存在“所有 agent 抢同一个 inbox”的问题；真正共享的是消息总线 BUS。

3. Lead 侧：consume_lead_inbox() 统一消费入口

本次讨论的第一段核心代码是：

```
def consume_lead_inbox(route_protocol: bool = True) -> list[dict]:
    msgs = BUS.read_inbox("lead")
    if not msgs:
        return []
    if route_protocol:
        for msg in msgs:
            meta = msg.get("metadata", {})
            req_id = meta.get("request_id", "")
            msg_type = msg.get("type", "")
            if req_id and msg_type.endswith("_response"):
                approve = meta.get("approve", False)
                match_response(msg_type, req_id, approve)
    return msgs
```

- **为什么需要统一入口：** `BUS.read\_inbox("lead")` 很可能是“读完清空”。如果某处直接读走 response 却没有调用 `match\_response()`，状态机就不会更新。
- **consume 函数做什么：** 读 Lead 的 inbox；识别带 request_id 的 `\_response` 消息；调用 `match\_response()` 进行协议路由；最后仍返回原始消息。
- **真正改状态的是谁：** `consume\_lead\_inbox()` 不直接改 `state.status`，它只是触发 `match\_response()`；真正的状态更新在 `match\_response()` 内。

隐患： `approve = meta.get("approve", False)` 会把缺失 approve 字段的响应默认当成 reject。更严谨做法是发现缺字段就忽略或报错。

4. Teammate 侧：handle_inbox_message() 处理协议消息

teammate 自己的循环里也会读 inbox，但它处理的是发给 teammate 的消息。核心分发函数：

```
def handle_inbox_message(name: str, msg: dict, messages: list) -> bool:
    msg_type = msg.get("type", "message")
    meta = msg.get("metadata", {})
    req_id = meta.get("request_id", "")

    if msg_type == "shutdown_request":
        BUS.send(name, "lead", "Shutting down gracefully.",
                  "shutdown_response",
                  {"request_id": req_id, "approve": True})
        return True

    if msg_type == "plan_approval_response":
        approve = meta.get("approve", False)
        if approve:
            messages.append({"role": "user",
                             "content": "[Plan approved] Proceed with the task."})
        else:
            messages.append({"role": "user",
                             "content": f"[Plan rejected] Feedback: {msg['content']}"})

    return False
```

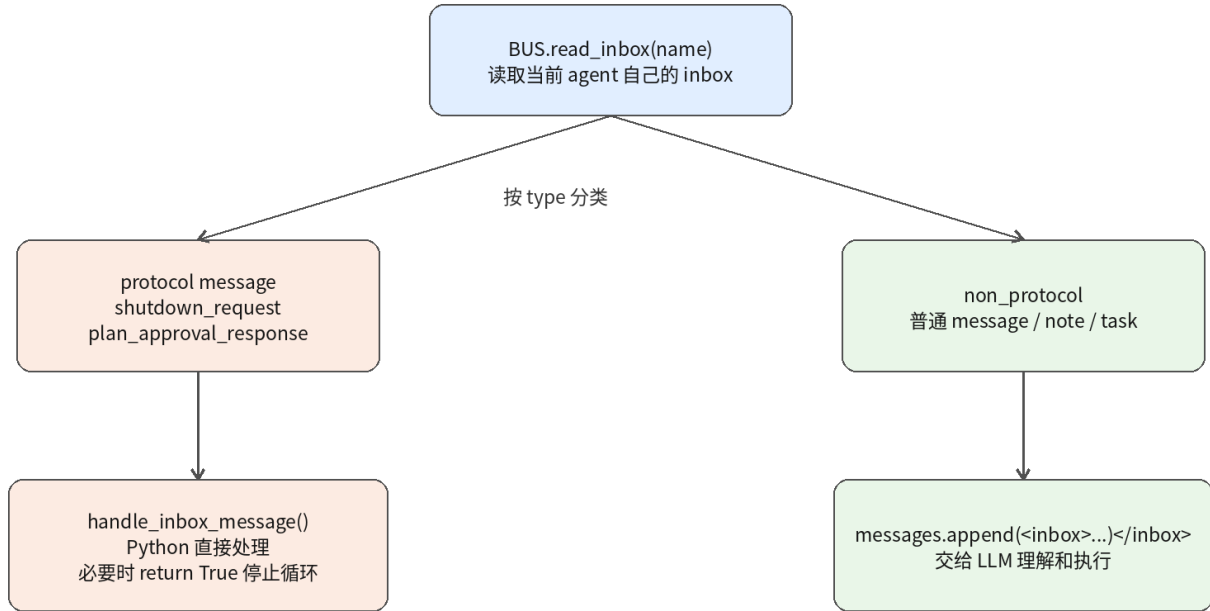
teammate 侧常见协议消息含义：

消息类型	方向	Python 层动作	返回值
shutdown_request	Lead -> teammate	回复 shutdown_response，带 回 request_id 和 approve=True	True，停止 teammate loop
plan_approval_response	Lead -> teammate	把 approved/rejected 结果 注入 messages，让 LLM 继续判断	False，继续运行

5. non_protocol：不是协议的新概念，只是普通消息暂存列表

你问“为什么又有 non_protocol 和上面出现过的东西”。关键是：inbox 里可能混着协议消息和普通消息。

思维导图 2：inbox 消费与消息分流



```
shutdown_requested = False
while not shutdown_requested:
    inbox = BUS.read_inbox(name)
    should_stop = False
    non_protocol = []
    for msg in inbox:
        if msg.get("type") in ("shutdown_request", "plan_approval_response"):
            should_stop = handle_inbox_message(name, msg, messages)
            if should_stop:
                break
        else:
            non_protocol.append(msg)

    if should_stop:
        shutdown_requested = True
        break

    if non_protocol:
        inbox_json = json.dumps(non_protocol)
        messages.append({"role": "user",
            "content": "<inbox>" + inbox_json + "</inbox>"})

# LLM turn
```

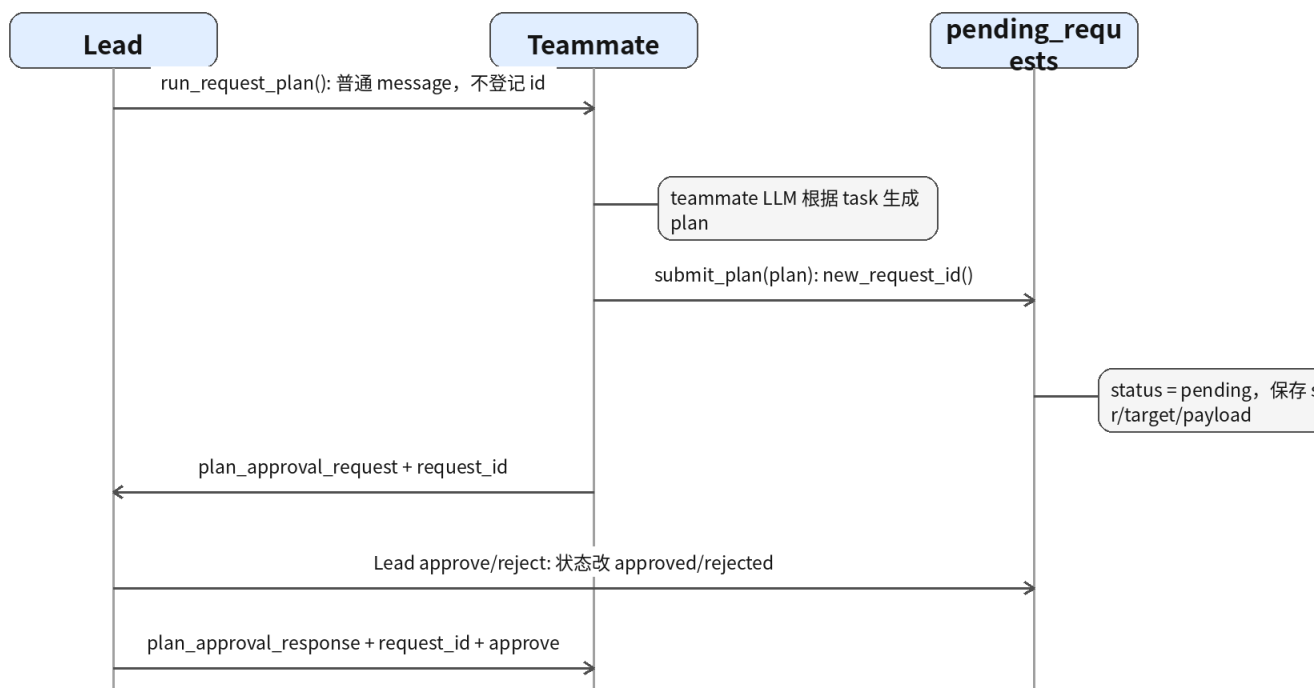
- **协议消息：**如 `shutdown_request`、`plan_approval_response`，由 Python 代码直接处理，因为它们会改变运行控制或协议状态。
- **普通消息：**如 `message`、`note`、任务说明，Python 不知道怎么执行，应包装成 `<inbox>...</inbox>` 交给 LLM。

- **为什么不能全交给 LLM：**例如 shutdown_request 如果只是塞给 LLM，模型可能口头答应关闭，但程序循环不会真的停止。

6. Plan Approval: request_plan 和 submit_plan 是两层

本次最重要的流程是“Lead 要求 teammate 提交计划”和“teammate 提交计划给 Lead 审批”。它们不是同一层。

流程图：Plan Approval 请求-响应链路



6.1 run_request_plan() 为什么不需要 request_id

```

def run_request_plan(teammate: str, task: str) -> str:
    """Lead asks a teammate to submit a plan for a task."""
    BUS.send("lead", teammate, f"Please submit a plan for: {task}",
            "message")
    return f"Asked {teammate} to submit a plan"
  
```

解释：`run\_request\_plan()` 只是 Lead 给 teammate 发普通 message：“请你为 task 提交计划”。它不登记 pending_requests，也不等待某个固定 response，因此教学版里不需要 request_id。

- `message` 表示普通消息，不是 `xxx\_request` 协议请求。
- 这条消息会进入 teammate 的 `non\_protocol`，再被包装进 `

- 如果你想追踪“Lead 要求提交计划”本身是否被确认，也可以把它升级成`plan\_request`协议，并引入`parent\_request\_id`，但教学版会更复杂。

6.2 _teammate_submit_plan(): 真正的协议级 request

```
def _teammate_submit_plan(from_name: str, plan: str) -> str:
    req_id = new_request_id()
    pending_requests[req_id] = ProtocolState(
        request_id=req_id, type="plan_approval",
        sender=from_name, target="lead",
        status="pending", payload=plan)
    BUS.send(from_name, "lead", plan,
             "plan_approval_request",
             {"request_id": req_id})
    return f"Plan submitted ({req_id}). Waiting for approval..."
```

- **plan 从哪里来：**不是 Lead 直接传入，而是 teammate 的 LLM 根据 inbox 里的 task 自己生成 plan，然后通过`submit\_plan(plan)`工具参数传入。
- **为什么这里需要 id：**因为它后面明确等待 Lead 的`plan\_approval\_response`，必须靠 request_id 对账。
- **pending 在哪里产生：**这一行`ProtocolState(..., status="pending", payload=plan)`把请求登记为待审批。
- **这不是硬拦截：**函数只是协议层请求，teammate 线程不会真的阻塞；如果要强制阻止 bash/write，需要在 tool_dispatch 层做 gating。

7. 状态机：pending 何时变成 approved / rejected

重点：`pending`是发起协议请求时写入的；`approved/rejected`是审批工具或`match\_response()`处理响应时写入的。

```
# 创建请求时:
pending_requests[req_id] = ProtocolState(..., status="pending")

# 审批或匹配响应时:
state.status = "approved" if approve else "rejected"
```

你问过下面这段是不是“考虑时差才重复判断”：

```
if not state:
    return f"Request {request_id} not found"
if state.status != "pending":
    return f"Request {request_id} already {state.status}"
state.status = "approved" if approve else "rejected"
```

- **不是时区意义的时差：**它不是在处理系统时间或时区。
- **是异步时序防御：**它防止 request_id 不存在、旧消息晚到、LLM 重复审批、同一个请求被 approve 后又被 reject。
- **本质是幂等性保护：**只有 `pending` 状态的请求可以被处理；已处理请求不能重复改状态。

8. shutdown_request 与 consume_lead_inbox 的方向差异

两个方向容易混淆，建议分成两条链：

- **Plan approval 链：**teammate -> Lead: `plan\_approval\_request`；Lead -> teammate: `plan\_approval\_response`。状态通常由 Lead 的审批函数直接改成 approved/rejected。
- **Shutdown 链：**Lead -> teammate: `shutdown\_request`；teammate -> Lead: `shutdown\_response`。Lead 侧 `consume\_lead\_inbox()` 读到 response 后调用 `match\_response()` 更新状态。

```
# shutdown 方向的关键点：
teammate 收到 shutdown_request
-> BUS.send(name, "lead", ..., "shutdown_response", {"request_id": req_id, "approve": True})

Lead 之后 consume_lead_inbox()
-> match_response("shutdown_response", req_id, True)
-> pending -> approved
```

9. continue 的作用范围

规则：`continue` 永远只作用于它所在的最近一层 `for` 或 `while`。看缩进即可判断。

```
for i in range(3):
    if i == 1:
        continue          # 属于外层 for，跳过本轮 i，内层 for 不会执行

    for j in range(2):
        print(i, j)
```

如果 `continue` 在内层 for 里面，则只跳过当前这轮内层循环，不影响外层循环进入下一次。

10. 本次代码设计的主要隐患与改进建议

位置 / 现象	风险	建议
直接 BUS.read_inbox()	读走响应但没有路由协议，pending_requests 不更新	所有 Lead 读取入口统一改为 consume_lead_inbox()
msg_type.endswith("_response")	过宽，debug_response/heartbeat_response 也可能误入协议路由	维护 PROTOCOL_RESPONSES 白名单

approve 默认 False	缺字段被当成 reject	缺少 approve 字段时跳过或报错
submit_plan 只是协议层请求	teammate 可能没等 approval 就继续调用工具	在 tool_dispatch 层加 code-level gating
无锁修改 pending_requests	多线程时可能两个线程同时处理同一请求	真正并发时使用 lock 或队列单线程化
run_request_plan 不追踪	无法证明 teammate 是否为这次 task 提交了对应 plan	需要时升级为 plan_request + parent_request_id

11. 最终记忆框架

一句话记忆： 协议消息走 Python 状态机，普通消息走 LLM 上下文。

- **MessageBus：** 负责把消息送到目标 agent 的 inbox。
- **inbox：** 不是全局公共列表，而是每个 agent 一个独立收件箱。
- **request_id：** 负责把 response 对回原 request。
- **pending_requests：** 负责保存请求的状态、发起者、目标、payload。
- **consume_lead_inbox：** Lead 侧“读取 + 对账”的统一入口。
- **handle_inbox_message：** teammate 侧协议消息分发器。
- **non_protocol：** 普通消息暂存列表，最终包装进 ``<inbox>`` 交给 LLM。
- **run_request_plan：** 普通通知，不登记 id。
- **_teammate_submit_plan：** 真正协议请求，生成 id，登记 pending，发送 request。

12. 推荐的脑内流程图

```

Lead: run_request_plan(task) -> 普通 message 给 teammate
Teammate: non_protocol -> <inbox> -> LLM 生成 plan
Teammate: submit_plan(plan) -> new_request_id()
System: pending_requests[req_id] = pending
System: BUS.send(plan_approval_request, request_id)
Lead: approve/reject -> status = approved/rejected
Lead: BUS.send(plan_approval_response, request_id, approve)
Teammate: handle_inbox_message(response) -> 注入 messages -> LLM 继续

```